

SOFTWARE REQUIREMENTS
SPECIFICATION

WINTER ARC

Personal Transformation & Productivity Platform

Document Type	Software Requirements Specification (SRS)
Version	1.0
Status	Baseline / Development Specification
Date	01 October 2026
Target Platform	Responsive Web Application
Primary Stack	Python + Django + PostgreSQL + HTML/CSS/JavaScript
Future Stack	Django REST Framework + React + AI/ML

Purpose: Define the functional, non-functional, technical, data, security, interface, testing, and deployment requirements for Winter Arc.

Confidentiality: Project documentation

Document Control

Item	Specification
Document ID	WA-SRS-001
Project Name	Winter Arc
Version	1.0
Prepared For	Academic / Portfolio Full-Stack Development Project
Primary Users	Individual users pursuing structured personal goals
System Type	Web-based personal productivity and self-management platform
Requirement Baseline	Requirements in this document define the initial production-oriented scope
Priority Scheme	MUST = mandatory; SHOULD = desirable; MAY = optional/future

Revision History

Version	Date	Change
1.0	01 Oct 2026	Initial complete SRS baseline

Table of Contents

1	1. Introduction
2	2. Product Overview
3	3. Stakeholders and User Classes
4	4. Product Scope
5	5. Assumptions, Constraints and Dependencies
6	6. Functional Requirements
7	7. Detailed Module Requirements
8	8. User Interface and UX Requirements
9	9. System Architecture
10	10. Database Requirements and Data Model
11	11. API Requirements
12	12. Security and Privacy Requirements
13	13. Non-Functional Requirements
14	14. Business Rules
15	15. Validation and Error Handling
16	16. Testing and Quality Assurance
17	17. Deployment and DevOps Requirements
18	18. Monitoring, Logging and Maintenance
19	19. Acceptance Criteria
20	20. MVP and Development Phases
21	21. Future Enhancements
22	22. Traceability and Requirement IDs
23	23. Risks and Mitigations
24	24. Glossary

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the complete requirements for Winter Arc, a responsive web application designed to help users organize a defined period of personal improvement through goals, habits, tasks, study/workout tracking, journaling, analytics, reminders, and gamification. The specification is intentionally implementation-aware so it can guide both academic development and a portfolio-quality production build.

1.2 Problem Statement

People commonly set improvement goals but lack a single system that connects long-term goals with daily execution and measurable outcomes. Winter Arc addresses this gap by combining planning, daily execution, reflection, progress measurement, and feedback in one application.

1.3 Objectives

- Provide a structured framework for a user-defined transformation period.
- Convert high-level goals into measurable tasks and recurring habits.
- Provide a daily dashboard showing what matters today.
- Capture study, fitness, productivity, mood/energy, and reflection data without making any one category mandatory.
- Provide historical analytics so users can identify patterns in their own behavior.
- Use gamification to reinforce consistent execution without making rewards the primary purpose.
- Provide a secure backend and normalized relational database.
- Expose REST APIs so the frontend can later be replaced or expanded with React.
- Create an architecture that can accommodate future AI/ML recommendations.

1.4 Intended Audience

- Student/developer implementing the project.
- Project evaluator or academic supervisor.
- Future contributors maintaining the codebase.
- End users using the application for personal planning and tracking.

1.5 Document Conventions

Requirements use identifiers such as WA-AUTH-001. Priority terms **MUST**, **SHOULD**, and **MAY** indicate implementation priority. A **MUST** requirement is part of the acceptance baseline unless explicitly deferred to a later phase.

2. Product Overview

2.1 Product Vision

Winter Arc is a personal transformation operating system: the user defines an arc, chooses measurable outcomes, executes daily actions, records relevant data, reviews progress, and adjusts the plan.

2.2 Core User Journey

- 1 Create an account and configure a profile.
- 2 Create a Winter Arc with a name, start date, end date, objective, and optional theme.
- 3 Create goals and divide them into milestones, habits, and tasks.
- 4 Define recurring habits and schedules.
- 5 Use the Today dashboard to execute tasks and habits.
- 6 Record study/workout sessions when applicable.
- 7 Write a daily journal entry and optionally record mood/energy.
- 8 Review weekly and monthly analytics.
- 9 Earn XP/achievements for completed actions.
- 10 Adjust goals and routines based on observed progress.

2.3 Product Principles

- Execution over decoration: dashboards must prioritize actionable information.
- User ownership: the user controls what is tracked and what is private.
- Measurability: progress should be based on stored events and explicit targets where possible.
- Privacy by default: personal journals and behavioral records are not public.
- Extensibility: API-first backend boundaries should permit a future React frontend and AI services.
- Graceful degradation: optional integrations must not prevent core task/habit tracking.

2.4 Technology Baseline

Layer	Technology	Purpose
Presentation	HTML5, CSS3, JavaScript; Bootstrap or Tailwind CSS	Responsive UI and client-side interactions
Backend	Python, Django	Application logic, routing, ORM, authentication
API	Django REST Framework	RESTful API for frontend/mobile/future clients
Database	PostgreSQL	Transactional relational storage
Background Jobs	Celery + Redis	Scheduled reminders and asynchronous processing
Charts	Chart.js or equivalent	Analytics visualization
Testing	pytest / Django Test Framework	Automated unit/integration tests
Deployment	Docker + Linux-based hosting	Reproducible deployment
Version Control	Git + GitHub	Source control and collaboration
Future AI	scikit-learn + optional LLM API	Personalized insights and recommendations

3. Stakeholders and User Classes

Stakeholder / Class	Role	Needs
End User	Uses the platform for personal transformation	Simple planning, tracking, privacy, useful feedback
Administrator	Operates the application	User management, system monitoring, moderation, audit visibility
Developer	Builds and maintains system	Clear requirements, modular architecture, tests, API contracts
Academic Evaluator	Reviews project	Traceable requirements, database design, security, testing, documentation
Notification Service	External/system component	Reliable delivery of configured reminders
Future AI Service	Optional intelligence layer	Aggregated, consented behavioral features; no unnecessary private text exposure

3.1 Access Roles

- USER: standard authenticated user; can access only their own personal records.
- ADMIN: system operator with restricted administrative functions.
- SUPERUSER: Django-level operational account; must not be used as a normal application identity.

4. Product Scope

4.1 In Scope

- Account registration, login, logout, password recovery and profile management.
- Winter Arc lifecycle management.
- Goals, milestones, tasks, habits, streaks and completion records.
- Daily dashboard and calendar.
- Study session and workout session tracking.
- Daily journal and optional mood/energy tracking.
- Progress dashboards, charts and summary analytics.
- XP, levels, badges and achievement events.
- Configurable reminders and notification preferences.
- REST API for core resources.
- Administrative functions, audit events and operational monitoring.
- Responsive design for desktop, tablet and mobile browsers.
- Production-oriented security, validation, testing and deployment.

4.2 Out of Scope for Initial Release

- Public social feed or follower system.
- Financial transaction processing.
- Medical diagnosis or clinical decision-making.
- Automatic wearable-device control.
- Unmoderated public sharing of journals.
- Fully autonomous AI coaching that makes consequential decisions.
- Native mobile applications; the web application remains the initial client.

4.3 Success Measures

Metric	Target Definition
Core completion	User can create an arc, goal, habit and task and record completion without errors.
Data integrity	No cross-user access to personal records; relational constraints prevent invalid ownership.
Performance	Typical authenticated pages respond within agreed NFR thresholds under expected project load.
Reliability	Background reminder failures are logged and retryable.
Testability	Critical business logic has automated tests and acceptance cases.
Maintainability	Application is divided into logical Django apps/services with documented API contracts.

5. Assumptions, Constraints and Dependencies

5.1 Assumptions

- Users have a modern browser and internet access for the hosted application.
- Dates and times are interpreted according to the user's configured timezone.
- Only the user can decide which personal metrics to track.
- Email/push delivery is an optional infrastructure dependency, not a prerequisite for core tracking.

5.2 Constraints

- The project is intended to be implementable by a student developer, so advanced infrastructure must remain incremental.
- Privacy-sensitive data requires strict authorization checks.
- AI features must not block the core application.
- Third-party services may impose quotas, costs, availability limits, and data-processing conditions.

5.3 Dependencies

Dependency	Purpose	Failure Behavior
PostgreSQL	Persistent data	Application enters controlled error state; no silent data loss
Redis	Caching/background queue	Core synchronous features continue where possible; queued jobs retry/fail visibly
Celery	Scheduled/background work	Reminder jobs retry and generate operational logs
Email provider	Email notifications	Notification failure logged; user can continue using app
Hosting provider	Application availability	Deployment/health checks identify outage
Optional LLM/AI provider	AI insights	AI module disabled gracefully if unavailable

6. Functional Requirements

6.1 Requirement Priority Legend

Priority	Meaning
MUST	Required for the applicable release and acceptance.
SHOULD	Important but can be deferred when schedule requires.
MAY	Optional enhancement.

6.2 High-Level Functional Requirements

ID	Requirement	Priority
WA-AUTH-001	System shall allow a user to register with a unique email/username and securely stored password.	MUST
WA-AUTH-002	System shall authenticate users and create a secure session/token according to the selected frontend architecture.	MUST
WA-AUTH-003	System shall support logout and invalidation of the active authentication context.	MUST
WA-AUTH-004	System shall provide password reset using a time-limited secure token.	MUST
WA-PROF-001	User shall be able to view and edit profile information and timezone.	MUST
WA-ARC-001	User shall be able to create, edit, pause and complete an Arc.	MUST
WA-GOAL-001	User shall be able to create measurable goals associated with an Arc.	MUST
WA-TASK-001	User shall be able to create, update, complete, reschedule and archive tasks.	MUST
WA-HAB-001	User shall be able to create recurring habits and record daily completion.	MUST
WA-DASH-001	System shall provide a Today dashboard combining due tasks, habits and progress.	MUST
WA-JRN-001	User shall be able to create, edit and delete private journal entries.	MUST
WA-ANA-001	System shall calculate and display progress metrics from stored records.	MUST
WA-GAM-001	System shall award XP for defined eligible completion events.	SHOULD
WA-NOT-001	System shall support configurable reminders for selected tasks/habits.	SHOULD
WA-API-001	Backend shall expose documented REST endpoints for core resources.	MUST
WA-ADM-001	Authorized administrators shall be able to manage operational user/system functions.	SHOULD
WA-AI-001	System may generate personalized insights from user-authorized activity data.	MAY

7. Detailed Module Requirements

7.1 Authentication & Account

ID	Requirement	Priority
WA-AUTH-010	Registration must validate email format, username rules, password policy, uniqueness, and required consent.	MUST
WA-AUTH-011	Passwords must never be stored in plaintext; Django's password hashing mechanisms must be used.	MUST
WA-AUTH-012	Repeated failed authentication attempts should be rate-limited or otherwise protected.	MUST
WA-AUTH-013	Password reset tokens must expire and be single-use.	SHOULD
WA-AUTH-014	Authenticated endpoints must enforce user ownership or role-based authorization.	SHOULD

7.2 Winter Arc Management

ID	Requirement	Priority
WA-ARC-010	An Arc shall contain name, description/objective, start date, end date, status, timezone and timestamps.	MUST
WA-ARC-011	Start date shall not be after end date.	MUST
WA-ARC-012	An Arc may be Draft, Active, Paused, Completed or Archived.	MUST
WA-ARC-013	Only one Arc may be marked as the user's primary active Arc at a time.	SHOULD
WA-ARC-014	Deleting an Arc must use an explicit confirmation and should prefer archival where historical data is valuable.	SHOULD

7.3 Goals & Milestones

ID	Requirement	Priority
WA-GOAL-010	Goal shall belong to exactly one user and may belong to one Arc.	MUST
WA-GOAL-011	Goal shall support title, description, category, priority, target value, unit, current value, deadline and status.	MUST
WA-GOAL-012	Progress shall be computed consistently from explicit goal measurements or linked completion events.	MUST
WA-GOAL-013	Milestones shall provide intermediate checkpoints and completion dates.	SHOULD

7.4 Tasks

ID	Requirement	Priority
WA-TASK-010	Task shall support title, description, due date/time, priority, status, optional goal, recurrence metadata and timestamps.	MUST
WA-TASK-011	Task status shall support at least Pending, In Progress, Completed and Cancelled.	MUST
WA-TASK-012	Completion must record a completion timestamp.	MUST
WA-TASK-013	Recurring tasks shall generate or represent future occurrences without duplicating historical completion data incorrectly.	SHOULD
WA-TASK-014	User shall be able to filter tasks by date, status, priority and goal.	SHOULD

7.5 Habits

ID	Requirement	Priority
WA-HAB-010	Habit shall support name, description, frequency, target count, active period and reminder preference.	MUST
WA-HAB-011	A habit completion record shall identify the habit, user, local date and completion quantity where applicable.	MUST
WA-HAB-012	The system shall calculate current and best streak using deterministic rules.	MUST
WA-HAB-013	Habit history shall remain available after a habit is archived.	SHOULD

7.6 Daily Check-in & Journal

ID	Requirement	Priority
WA-JRN-010	A user may create one daily check-in for a local calendar date.	MUST
WA-JRN-011	Check-in may include mood, energy, sleep duration, reflection and notes; fields should be optional.	MUST
WA-JRN-012	Journal entries shall be private by default.	MUST
WA-JRN-013	Journal content shall be excluded from analytics unless the user explicitly enables an analysis feature.	SHOULD

7.7 Study Tracking

ID	Requirement	Priority
WA-STU-010	User shall be able to record a study session with subject, start/end or duration, topic and notes.	MUST
WA-STU-011	System shall calculate daily, weekly and monthly study totals.	MUST
WA-STU-012	Study sessions may be linked to a goal.	MUST

7.8 Workout Tracking

ID	Requirement	Priority
WA-WKT-010	User shall be able to create workout sessions and record exercises.	MUST
WA-WKT-011	Exercise records may include sets, repetitions, duration, load and notes.	MUST
WA-WKT-012	System shall summarize workout frequency and selected measurable performance values.	MUST

7.9 Calendar

ID	Requirement	Priority
WA-CAL-010	Calendar shall display tasks, habits and relevant milestones using the user's timezone.	MUST
WA-CAL-011	Selecting an item shall open the corresponding detail/edit action.	MUST
WA-CAL-012	Calendar filtering shall allow category-based visibility.	MUST

7.10 Dashboard

ID	Requirement	Priority
WA-DASH-010	Dashboard shall display current Arc progress, today's tasks, habits, streaks and key metrics.	MUST
WA-DASH-011	Dashboard shall identify overdue items separately from upcoming items.	MUST

ID	Requirement	Priority
WA-DASH-012	Dashboard shall support a fast completion action without navigating through multiple screens.	MUST

7.11 Analytics

ID	Requirement	Priority
WA-ANA-010	System shall provide daily/weekly/monthly aggregation for task completion, habit consistency, study time and workouts.	MUST
WA-ANA-011	Analytics shall clearly distinguish raw values, derived metrics and user-entered targets.	MUST
WA-ANA-012	Date ranges and timezone must be applied consistently to aggregation.	MUST
WA-ANA-013	Analytics pages shall remain usable when data volume is low; empty states shall explain how to start collecting data.	SHOULD

7.12 Gamification

ID	Requirement	Priority
WA-GAM-010	XP rules shall be defined in a central service/module rather than duplicated in UI code.	MUST
WA-GAM-011	Achievements shall have deterministic unlock criteria.	MUST
WA-GAM-012	XP should not be awarded multiple times for the same unique completion event unless the rule explicitly permits it.	MUST
WA-GAM-013	Users shall be able to see the reason for an XP award.	SHOULD

7.13 Notifications

ID	Requirement	Priority
WA-NOT-010	User shall be able to configure reminder preferences.	MUST
WA-NOT-011	Notifications shall respect user timezone and quiet-hour settings.	MUST
WA-NOT-012	Failed background notification jobs shall be retried according to configured retry policy.	MUST
WA-NOT-013	The system shall avoid sending duplicate notifications for the same scheduled event.	SHOULD

8. User Interface and UX Requirements

8.1 Design Requirements

- Responsive layout must work on desktop, tablet and mobile viewport sizes.
- Primary navigation should expose Dashboard, Tasks, Habits, Goals, Calendar, Journal, Analytics and Profile.
- Important actions such as completing a task or habit must be visually prominent.
- Forms must show field-level validation and preserve valid input when another field fails.
- Destructive actions must require confirmation.
- Loading, empty, success and error states must be designed rather than left blank.
- Charts must include labels or accessible summaries so information is not conveyed by color alone.
- Typography, spacing, buttons, forms and cards should use a consistent design system.

8.2 Primary Screens

Screen	Purpose	Core Components
Landing	Explain product and provide authentication entry	Hero, benefits, CTA
Register/Login	Account access	Form, validation, recovery
Onboarding	Create initial Arc and first goals	Wizard/progress steps
Dashboard	Daily command center	Arc progress, tasks, habits, metrics
Tasks	Task management	List, filters, creation/edit form
Habits	Routine tracking	Habit cards, streaks, history
Goals	Long-term outcomes	Goal cards, progress, milestones
Calendar	Time-based overview	Calendar grid, filters
Journal	Reflection	Date selector, editor, check-in
Study	Study tracking	Sessions, subjects, totals
Workout	Fitness tracking	Sessions, exercises, progress
Analytics	Trend analysis	Charts, date range, summaries
Achievements	Gamification	XP, levels, badges
Profile/Settings	Personal configuration	Profile, timezone, notification settings
Admin	Operations	Users, audit events, system status

8.3 Accessibility

- All interactive controls must be keyboard reachable.
- Form controls require visible labels.
- Color contrast must be adequate for normal text and controls.
- Focus state must remain visible.
- Images require meaningful alt text where informational.
- Semantic HTML should be used before ARIA attributes.

9. System Architecture

9.1 Logical Architecture

The recommended architecture is a modular Django monolith with a REST API boundary. This provides lower operational complexity than microservices while retaining clean separation for future scaling.

Layer	Responsibilities
Client/UI	Responsive HTML/CSS/JS initially; optional React SPA later
Presentation/API	Django views, serializers, request validation, authentication
Application Services	Goal progress, streak calculation, XP, analytics aggregation, notification orchestration
Domain/Models	Django ORM models and relationships
Infrastructure	PostgreSQL, Redis, Celery, email provider, object storage if needed
Observability	Structured logs, health checks, error tracking, audit events

9.2 Recommended Django App Boundaries

Django App	Responsibility
accounts	User profile, authentication extensions, preferences
arcs	Winter Arc lifecycle and configuration
goals	Goals and milestones
tasks	Tasks and recurrence
habits	Habits, completion events, streaks
journal	Daily check-ins and private journal
study	Subjects and study sessions
workouts	Workout sessions and exercise records
analytics	Derived metrics and reporting
gamification	XP, levels, achievements
notifications	Preferences, scheduling, delivery logs
api	REST routing, serializers and API versioning
core	Shared utilities, audit, health checks and common services

9.3 Architectural Rules

- Business rules must not depend on template/UI code.
- Database queries should be centralized in model managers/querysets or services where complexity warrants it.
- External service calls must be isolated behind service interfaces.
- Critical calculations such as streaks and XP must have deterministic automated tests.
- API responses should use stable resource representations and explicit validation errors.

10. Database Requirements and Data Model

10.1 Database Technology

PostgreSQL is the recommended production database because the application is relational, transaction-oriented, and expected to perform aggregate queries over structured activity data.

10.2 Core Entities

Entity	Key Fields / Relationships
User	id, email/username, password hash, created_at, last_login
Profile	user_id, display_name, avatar, timezone, bio, preferences
Arc	id, user_id, name, objective, start_date, end_date, status, is_primary
Goal	id, user_id, arc_id, title, category, target, current_value, unit, deadline, status
Milestone	id, goal_id, title, target_value, due_date, completed_at
Task	id, user_id, goal_id, title, due_at, priority, status, recurrence_rule
Habit	id, user_id, name, frequency, target_count, active_from, active_until
HabitCompletion	id, habit_id, local_date, quantity, completed_at
JournalEntry	id, user_id, local_date, mood, energy, sleep_hours, content
StudySession	id, user_id, goal_id, subject, started_at, ended_at, duration
WorkoutSession	id, user_id, goal_id, started_at, ended_at, notes
ExerciseRecord	id, workout_id, exercise_name, sets, reps, load, duration
Achievement	id, code, name, description, xp_reward, criteria_version
UserAchievement	id, user_id, achievement_id, unlocked_at
XPEvent	id, user_id, source_type, source_id, amount, created_at
NotificationPreference	user_id, channel, enabled, quiet_start, quiet_end
NotificationJob	id, user_id, type, scheduled_for, status, attempts
AuditEvent	id, actor_user_id, action, entity_type, entity_id, timestamp, metadata

10.3 Data Integrity Rules

- Every personal entity must have a clear ownership path to a user.
- Foreign keys must use deliberate deletion behavior; historical records should generally be protected from accidental cascading deletion.
- Unique constraints should prevent duplicate daily habit completion records where the business rule allows only one.
- Indexes should exist on common filters such as user_id, due_at, local_date, status and created_at.
- Transactions must be used where multiple writes must succeed or fail together, especially completion + XP event operations.

10.4 Data Retention

The system should retain user records until the user requests deletion or the application's documented retention policy requires removal. Deletion workflows must distinguish active deletion from archival where necessary for operational integrity.

11. API Requirements

11.1 API Principles

- API versioning should begin at /api/v1/.
- JSON is the default request/response representation.
- Authentication and authorization are mandatory for personal resources.
- Validation errors must return structured, actionable messages.
- Pagination must be used for potentially large collections.
- API documentation should be generated/maintained using OpenAPI-compatible tooling.

11.2 Representative Endpoint Contract

Method	Endpoint	Purpose
POST	/api/v1/auth/register/	Register user
POST	/api/v1/auth/login/	Authenticate user
POST	/api/v1/auth/logout/	Terminate authentication context
POST	/api/v1/auth/password-reset/	Start password reset
GET	/api/v1/profile/	Get current profile
PATCH	/api/v1/profile/	Update current profile
GET	/api/v1/arcs/	List user's arcs
POST	/api/v1/arcs/	Create arc
GET	/api/v1/arcs/{id}/	Get arc
PATCH	/api/v1/arcs/{id}/	Update arc
DELETE	/api/v1/arcs/{id}/	Archive/delete arc according to policy
GET	/api/v1/goals/	List goals
POST	/api/v1/goals/	Create goal
GET	/api/v1/tasks/	List/filter tasks
POST	/api/v1/tasks/	Create task
POST	/api/v1/tasks/{id}/complete/	Complete task
GET	/api/v1/habits/	List habits
POST	/api/v1/habits/{id}/complete/	Record habit completion
GET	/api/v1/dashboard/	Return dashboard summary
GET	/api/v1/analytics/	Return analytics for selected date range
POST	/api/v1/journal/	Create journal entry
GET	/api/v1/calendar/	Return calendar events

11.3 API Security

- Never trust user_id values supplied by clients to determine ownership.
- Resolve ownership from the authenticated principal.

- Use server-side authorization on every object endpoint.
- Apply rate limiting to authentication and other abuse-prone endpoints.
- Do not return password hashes, reset tokens, internal secrets or unnecessary private metadata.

12. Security and Privacy Requirements

12.1 Authentication Security

- Use Django's proven password hashing framework.
- Use HTTPS in production.
- Secure cookies must use appropriate Secure, HttpOnly and SameSite settings.
- CSRF protection must remain enabled for cookie-authenticated browser workflows.
- Authentication endpoints should be rate-limited.
- Secrets must be supplied through environment/secret management rather than committed to source control.

12.2 Authorization

- Object-level authorization is mandatory for all user-owned records.
- An administrator must not automatically receive unrestricted access to private journal content unless a documented operational requirement exists.
- API permissions and web permissions must enforce the same ownership rules.

12.3 Input and Application Security

- Validate all server-side input even when client-side validation exists.
- Use ORM parameterization; do not construct raw SQL from untrusted input.
- Escape or sanitize rich text according to the editor's supported HTML policy.
- Prevent insecure direct object reference by checking ownership.
- Configure security headers and production Django security settings.
- File uploads, if introduced, must validate type, size and storage location.

12.4 Privacy

- Journal entries, mood/energy values and behavioral records are private by default.
- Users should be able to export their personal data in a documented format.
- Users should be able to request account/data deletion according to the application's retention policy.
- AI analysis must be opt-in when it requires sending data to an external provider.
- Only the minimum data required for a feature should be sent to external services.

12.5 Threat Model Summary

Threat	Mitigation
Credential theft	Strong hashing, HTTPS, rate limiting, secure reset flow
Broken object authorization	Server-side ownership checks and tests
SQL injection	Django ORM/parameterized queries
XSS	Template escaping, sanitized rich text, CSP where practical
CSRF	Django CSRF protection and secure cookie configuration
Abuse/rate attacks	Rate limits, request validation, monitoring
Data leakage	Least privilege, private defaults, logging discipline

Threat	Mitigation
Secret exposure	Environment secrets; never commit credentials
Background-job duplication	Idempotency keys/business event uniqueness

13. Non-Functional Requirements

ID	Category	Requirement	Priority
WA-NFR-001	Performance	Typical dashboard/API requests should target a server response time under 500 ms at the application layer under expected project load, excluding slow external services.	SHOULD
WA-NFR-002	Availability	Production deployment should include health checks and restart/recovery procedures.	SHOULD
WA-NFR-003	Scalability	Architecture should permit horizontal web-worker scaling without changing core domain logic.	SHOULD
WA-NFR-004	Maintainability	Code shall use clear module boundaries, naming conventions, documentation and automated tests.	MUST
WA-NFR-005	Security	Production traffic shall use HTTPS and authentication secrets shall not be stored in source control.	MUST
WA-NFR-006	Usability	A first-time user should be able to create an Arc and first task without external instructions.	SHOULD
WA-NFR-007	Accessibility	Core workflows shall be keyboard usable and provide labels and visible focus.	MUST
WA-NFR-008	Compatibility	Application shall support current major versions of Chrome/Edge/Firefox/Safari where practical.	SHOULD
WA-NFR-009	Reliability	Critical database operations shall be transactional and recoverable from application errors.	MUST
WA-NFR-010	Observability	Application shall provide structured logs and health endpoints for production diagnosis.	SHOULD
WA-NFR-011	Backup	Production database shall have a documented backup and restore procedure.	MUST
WA-NFR-012	Privacy	Personal records shall be isolated by authenticated user ownership.	MUST
WA-NFR-013	Localization	Dates/times shall be rendered using user timezone; locale expansion may be added later.	SHOULD
WA-NFR-014	Testability	Critical domain services and authorization paths shall be covered by automated tests.	MUST

13.1 Performance Measurement Notes

Performance targets are engineering targets, not guarantees. They must be measured under a defined test dataset and load profile. External email, AI, or third-party latency should be measured separately from core application latency.

13.2 Scalability Strategy

- Start with a modular monolith.
- Add indexes based on measured query patterns.
- Use pagination for large collections.
- Move expensive analytics/notifications to background jobs.
- Introduce caching only where measurements justify it.
- Separate read-heavy analytics workloads from transactional writes if the system later reaches sufficient scale.

14. Business Rules

Rule ID	Area	Rule
BR-001	Arc dates	start_date <= end_date.
BR-002	Primary Arc	A user may have at most one primary active Arc.
BR-003	Task ownership	A task can only be viewed/modified by its owner or authorized system role.
BR-004	Habit completion	A completion event must belong to the habit owner and valid local date.
BR-005	Streak	Streak calculation must use a documented frequency-aware algorithm.
BR-006	XP idempotency	A unique eligible event should not award duplicate XP unless explicitly designed.
BR-007	Goal progress	Progress must use a documented target/current-value calculation and must not silently mix incompatible units.
BR-008	Timezone	Daily records are evaluated using the user's configured timezone.
BR-009	Journal privacy	Journal entries are private by default and excluded from public endpoints.
BR-010	Deletion	Destructive deletion must respect foreign-key integrity and retention policy.
BR-011	Notifications	Notifications must honor enabled channels and quiet hours.
BR-012	Analytics	Derived metrics must be reproducible from source records or documented snapshots.

14.1 Streak Definition

For an initial implementation, a habit streak represents the number of consecutive expected habit periods completed according to the habit's configured frequency. The exact algorithm must be implemented as a single tested service so that weekly, selected-day, and target-count habits do not produce inconsistent results.

14.2 Progress Definition

Goal progress should be represented as a percentage only when a numeric target and compatible current value exist. Otherwise the UI should show milestone/completion status rather than inventing a percentage.

15. Validation and Error Handling

15.1 Validation Rules

- Required fields must be checked server-side.
- Dates must use valid calendar values.
- End dates must not precede start dates.
- Numeric targets must respect configured minimum/maximum constraints where applicable.
- Text fields must have documented maximum lengths.
- Enum values must be validated against server-defined choices.
- Recurrence rules must be validated before persistence.
- Uploaded files, if supported, must be checked for size and allowed content types.

15.2 Error Response Format

API errors should use a stable structure containing an error/code field and field-level details where relevant. Example conceptual response: `{'code': 'VALIDATION_ERROR', 'message': 'Please correct the highlighted fields.', 'fields': {'due_at': ['Invalid date/time.']}}`.

15.3 User-Facing Error States

Condition	Expected UI Behavior
Validation failure	Show specific field errors and preserve valid input.
Unauthorized	Redirect to login or return 401 depending on client.
Forbidden	Show access-denied state without revealing private resource existence unnecessarily.
Not found	Show a clear not-found state.
Server error	Show generic message; log technical details server-side.
Network failure	Show retry action and avoid silently losing entered data.
Background job failure	Record failure and retry where configured.

16. Testing and Quality Assurance

16.1 Testing Strategy

Test Level	Scope	Examples
Unit	Pure functions/services	XP calculation, streak calculation, goal progress
Model	Constraints/relationships	Ownership, uniqueness, date rules
API	Endpoint contracts	Auth, CRUD, filtering, permissions
Integration	Cross-module workflows	Complete task + XP + analytics
UI/E2E	Critical user journeys	Register → create Arc → complete task → dashboard
Security	Abuse and authorization	IDOR, CSRF, auth rate limiting
Performance	Measured workload	Dashboard, analytics, list queries
Regression	Previously fixed defects	Automated reproduction cases

16.2 Minimum Critical Test Cases

- 1 User A cannot retrieve User B's task by changing an ID.
- 2 User cannot create a task attached to another user's goal.
- 3 Password reset token expires and cannot be reused.
- 4 Arc cannot be created with an end date before its start date.
- 5 Completing the same task twice does not create duplicate completion/XP events.
- 6 Habit streak changes correctly when an expected period is missed.
- 7 Daily check-in uses the user's timezone rather than server UTC date alone.
- 8 Dashboard excludes another user's data.
- 9 Notification job respects quiet hours.
- 10 Analytics for an empty date range returns a valid empty state rather than an error.
- 11 Admin-only endpoints reject standard users.
- 12 Deleted/archived records behave according to the documented retention policy.

16.3 Definition of Done

- Requirement implemented and linked to requirement ID.
- Automated tests exist for critical behavior.
- Authorization has been tested.
- UI has loading, success, empty and error states.
- Database migration is reproducible.
- API documentation is updated.
- Code passes linting/formatting and test suite.
- Deployment configuration is reproducible.

17. Deployment and DevOps Requirements

17.1 Environment Model

Environment	Purpose
Local	Developer implementation and debugging
Test/CI	Automated tests, migrations and quality checks
Staging	Production-like validation before release
Production	Live user-facing application

17.2 Production Components

- Web application process running Django.
- PostgreSQL database.
- Redis service when background jobs/caching are enabled.
- Celery worker and scheduler for asynchronous/scheduled work.
- Reverse proxy/load balancer with HTTPS.
- Static/media storage strategy.
- Monitoring and centralized logs.

17.3 CI/CD Pipeline

- 1 Push/merge to Git repository.
- 2 Install dependencies using locked/reproducible versions.
- 3 Run formatting/lint checks.
- 4 Run unit, API and integration tests.
- 5 Run security/dependency checks where available.
- 6 Build deployment artifact/container.
- 7 Apply migrations in a controlled deployment step.
- 8 Deploy to staging and run smoke tests.
- 9 Promote to production after acceptance checks.
- 10 Monitor health and rollback when release causes a critical regression.

17.4 Environment Variables

Secrets and environment-specific configuration should include database credentials, Django secret key, allowed hosts, email credentials, Redis URL, AI provider keys and other external-service credentials. These values must not be committed to Git.

18. Monitoring, Logging and Maintenance

18.1 Health Checks

- Application health endpoint should verify basic process readiness.
- Readiness checks may verify database connectivity and required dependencies.
- Background worker health should be observable separately.

18.2 Logging

- Use structured logs with timestamp, severity, request/correlation ID and relevant component.
- Do not log passwords, authentication tokens, journal contents, or sensitive secrets.
- Log failed background jobs with enough metadata for diagnosis.
- Audit security-sensitive events such as login failures, password reset requests and administrative changes.

18.3 Backup and Recovery

- Automated database backups should be configured for production.
- Restore procedures must be tested rather than assumed to work.
- Recovery objectives should be documented when the system moves beyond academic/demo use.

18.4 Maintenance

- Dependencies should be updated regularly.
- Security advisories should be reviewed.
- Database migrations must remain version-controlled.
- Old/unused notification jobs and temporary data should be cleaned according to policy.

19. Acceptance Criteria

19.1 MVP Acceptance

Area	Acceptance Condition
Account	A new user can register, log in, log out and recover password.
Arc	User can create an Arc with valid dates and see its active state.
Goals	User can create a goal and see its progress.
Tasks	User can create, edit, complete and filter tasks.
Habits	User can create a habit and record daily completion/streak.
Dashboard	Today dashboard shows the authenticated user's relevant records.
Journal	User can create and retrieve private journal/check-in records.
Analytics	User can view at least basic completion/study/habit summaries.
Security	Cross-user object access is rejected.
Database	Migrations create all required tables and constraints.
Testing	Critical acceptance workflows have automated tests.
Deployment	Application can be deployed from documented configuration.

19.2 Release Gate

- No unresolved critical security defect.
- No known data-loss defect in core workflows.
- All MUST requirements for the target release are implemented or formally deferred.
- Automated test suite passes.
- Database migrations apply cleanly to a fresh environment.
- Production settings do not expose development/debug configuration.

20. MVP and Development Phases

20.1 Phase Plan

Phase	Deliverable	Main Skills
0	Project setup, Git, environment, architecture	Python, Git, Django setup
1	Authentication + profile	Django auth, forms, security
2	Arc + goals + milestones	Models, ORM, relationships
3	Tasks + habits + streaks	CRUD, business services, validation
4	Dashboard + calendar	Templates/JS, queries, UX
5	Journal + study + workout	Data modeling, forms, reporting
6	Analytics + charts	Aggregation, Chart.js, optimization
7	Gamification + notifications	Celery, Redis, event logic
8	REST API	DRF, serializers, permissions, OpenAPI
9	Security + tests	pytest/Django tests, threat cases
10	Docker + deployment	Containers, PostgreSQL, CI/CD
11	AI/ML extension	Feature engineering, recommendation models

20.2 Recommended MVP

The first deployable version should contain authentication, profile, Arc, goals, tasks, habits, dashboard, basic journal, PostgreSQL persistence, authorization, REST API basics, automated tests and deployment. Study/workout, advanced analytics, gamification, notifications and AI can be layered afterward.

20.3 Development Principle

Do not build the AI layer before the underlying event data is reliable. The quality of recommendations depends on clean, consistent historical data. The first engineering priority is therefore a trustworthy tracking system.

21. Future Enhancements

21.1 AI/ML Recommendation Engine

- Predict likelihood of completing a task based on historical execution patterns.
- Suggest realistic daily workload using historical completion rate.
- Identify recurring bottlenecks such as frequent overdue categories.
- Recommend habit scheduling based on observed completion patterns.
- Generate weekly summaries from structured metrics.
- Provide an explainable reason for each recommendation.

21.2 AI Safety/Privacy Requirements

- AI recommendations must be advisory, not authoritative.
- User consent must be obtained before sending private data to external AI services.
- Sensitive journal text should not be sent by default.
- Recommendations should cite the underlying behavioral signal where practical.
- Users must be able to disable AI features.

21.3 Additional Features

- Progress heatmaps.
- Import/export.
- Public challenge templates without exposing private records.
- Optional PWA/offline support.
- Native mobile client using the same API.
- Calendar provider integrations.
- Wearable integrations where privacy and API availability permit.
- Advanced cohort analytics only if a multi-user product is introduced.

22. Traceability and Requirement IDs

22.1 Requirement Naming Scheme

WA identifies the Winter Arc project. The middle code identifies the module (AUTH, ARC, GOAL, TASK, HAB, JRN, ANA, API, NFR, etc.). The numeric suffix uniquely identifies the requirement.

22.2 Traceability Matrix

Requirement Group	Primary Design Area	Primary Test Area
WA-AUTH-*	accounts + authentication	Authentication/security tests
WA-ARC-*	arcs	Arc lifecycle tests
WA-GOAL-*	goals	Goal/progress tests
WA-TASK-*	tasks	CRUD/ownership/completion tests
WA-HAB-*	habits	Frequency/streak tests
WA-JRN-*	journal	Privacy/CRUD tests
WA-STU-*	study	Duration/aggregation tests
WA-WKT-*	workouts	Session/exercise tests
WA-DASH-*	dashboard	Integration/UI tests
WA-ANA-*	analytics	Aggregation/timezone tests
WA-GAM-*	gamification	XP/idempotency tests
WA-NOT-*	notifications	Scheduler/retry tests
WA-API-*	api	Contract/permission tests
WA-NFR-*	cross-cutting	Performance/security/quality tests

22.3 Change Control

Any requirement change after baseline should record the changed requirement ID, reason, impact on database/API/UI, affected tests, and release phase. Schema changes must be accompanied by a migration.

23. Risks and Mitigations

Risk	Impact	Mitigation
Scope expansion	High	Freeze MVP; implement features by phases
Overengineering	Medium	Use modular monolith before microservices
Poor data model	High	Design relationships before UI; test constraints
Cross-user data leakage	Critical	Central ownership checks + security tests
AI built too early	Medium	Collect clean event data first
Notification complexity	Medium	Make notifications optional and background-driven
Time constraints	High	Prioritize MUST requirements and deploy incrementally
Third-party outage	Medium	Graceful failure and retries
Performance regression	Medium	Indexes, pagination, profiling and measured optimization
Security dependency vulnerabilities	High	Regular dependency updates and CI checks
Loss of database data	Critical	Automated backups and tested restore process

23.1 Highest-Risk Engineering Areas

The most important engineering risks are authorization, data integrity, recurrence/streak calculations, timezone handling, background job idempotency, and privacy. These areas deserve tests before visual polish.

24. Glossary

Term	Definition
Arc	A defined personal-improvement period with start/end dates and an objective.
Goal	A measurable or status-based outcome the user wants to achieve.
Milestone	An intermediate checkpoint within a goal.
Task	A discrete action that can be scheduled and completed.
Habit	A recurring behavior tracked over time.
Streak	A consecutive sequence of expected habit periods completed.
Check-in	A daily record containing optional self-reported metrics and reflection.
XP	Experience points awarded for defined eligible actions.
Achievement	A badge/unlock triggered by deterministic criteria.
REST API	HTTP-based interface exposing application resources and operations.
ORM	Object-Relational Mapping; Django ORM maps Python models to database tables.
JWT	JSON Web Token, commonly used for stateless API authentication.
CSRF	Cross-Site Request Forgery; an attack mitigated by request authenticity controls.
XSS	Cross-Site Scripting; injection of executable content into web pages.
Celery	Python task queue used for asynchronous/background jobs.
Redis	In-memory data store commonly used as cache or message broker.
Idempotency	Property where repeating an operation does not incorrectly duplicate its effect.
NFR	Non-Functional Requirement, such as performance, security or usability.
MVP	Minimum Viable Product; smallest release satisfying core user value.
AI/ML	Artificial Intelligence / Machine Learning functionality added after reliable core data collection.

Final Engineering Note

This SRS is a baseline specification. During implementation, technology-specific decisions—such as session authentication versus JWT for a particular client, exact recurrence syntax, hosting provider, email provider, charting library, and AI provider—should be documented as architecture decisions rather than silently changing the requirements. The core contract remains: secure ownership, reliable data, deterministic business rules, measurable progress, testable APIs, and a usable daily workflow.

End of Document